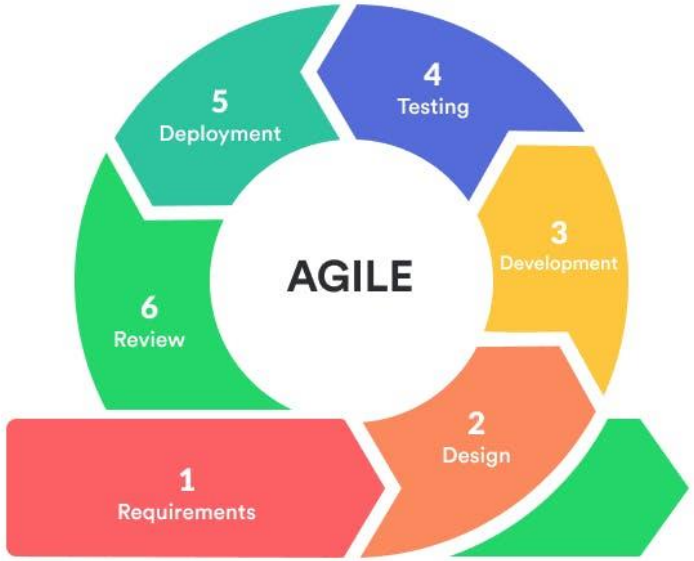
웹 개발자 부트캠프 과정

CodingOn

개발 문화

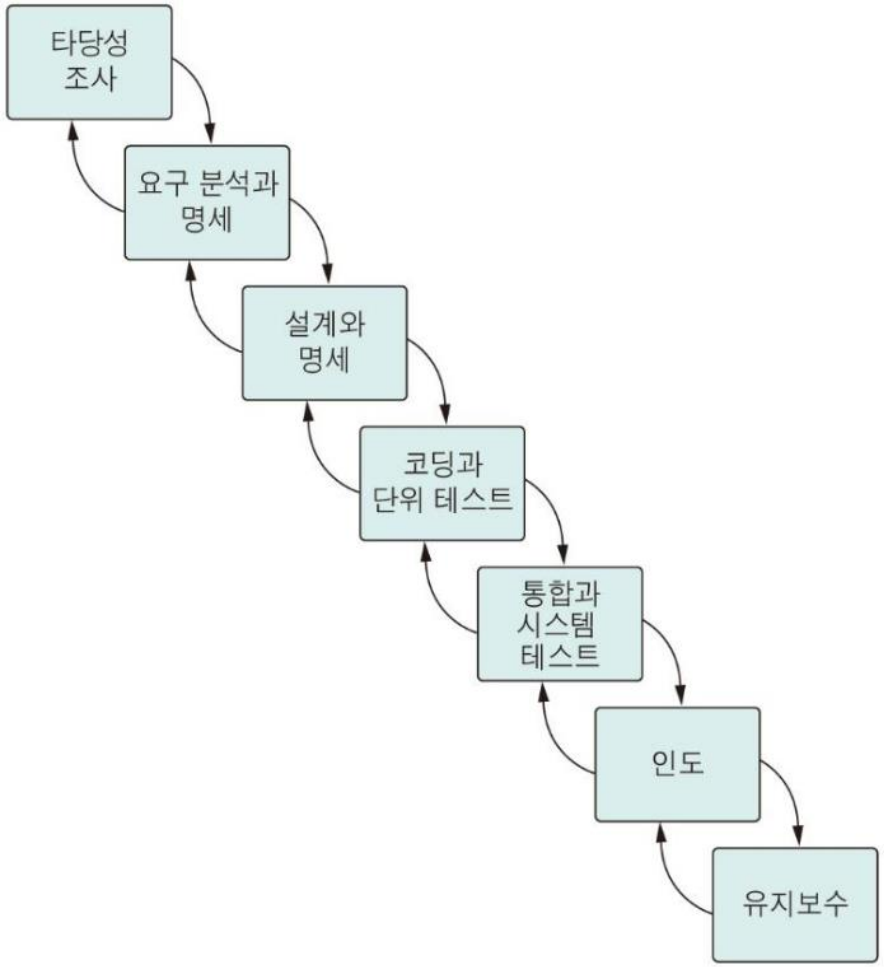
개발 문화

개발팀이나 조직이 어떻게 일하는 가?


AGILE
1
Requirements
2
Design
3
Development
4
Testing
5
Deployment
6
Review

Agile 방법론

Waterfall



Waterfall Model (폭포수 모델)

- 가장 익숙한 소프트웨어 개발 기법
- 고전적인 소프트웨어 생명 주기
- 병행 수행되지 않고 순차적으로 수행

Q. Waterfall Model의 장단점으로는 무엇이 있을까요?

Waterfall Model의 장단점

장점

- 단순한 모델이라 이해가 쉽다.
- 단계별로 정형화된 접근이 가능해 문서화가 가능하다.
- 프로젝트 진행 상황을 한눈에 명확하게 파악 가능하다.

단점

- 변경을 수용하기 어렵다.
- 시스템의 동작을 후반에 가야지만 확인이 가능하다.
- 대형 프로젝트에 적용하는 것이 부적합하고, 일정이 지연될 가능성이 크다.

Agile

agile 미국식 ['ædʒl] 영국식 ['ædʒaɪl]

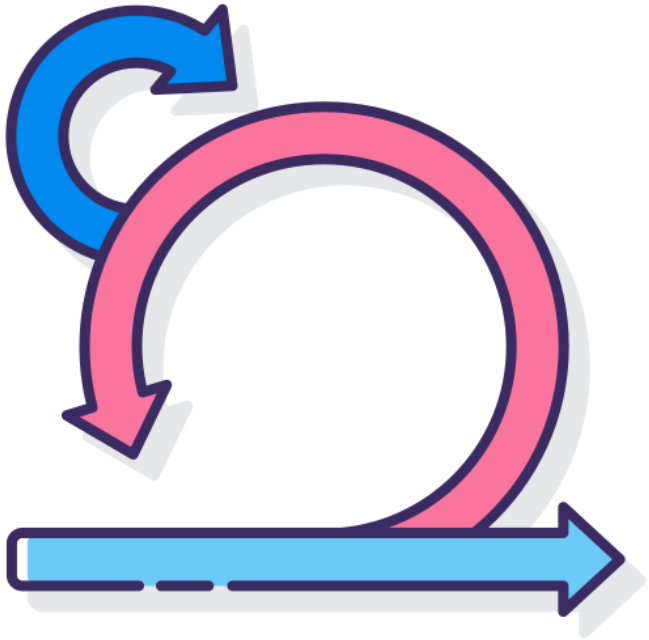
1. [형용사] 날렵한, 민첩한 (=nimble)
2. [형용사] (생각이) 재빠른, 기민한

- 짧은 주기의 개발 단위를 반복해 하나의 큰 프로젝트를 완성해 나가는 것
- 🔑 협력과 피드백
- 🔑 유연한 일 진행 + 빠른 변화 대응

Agile

- 짧은 주기로 설계, 개발, 테스트, 배포 과정을 반복
- 요구 사항을 작은 단위로 쪼개 그에 대한 솔루션을 만들고, 빠르게 보여줌으로써 요구 사항에 대한 검증을 진행

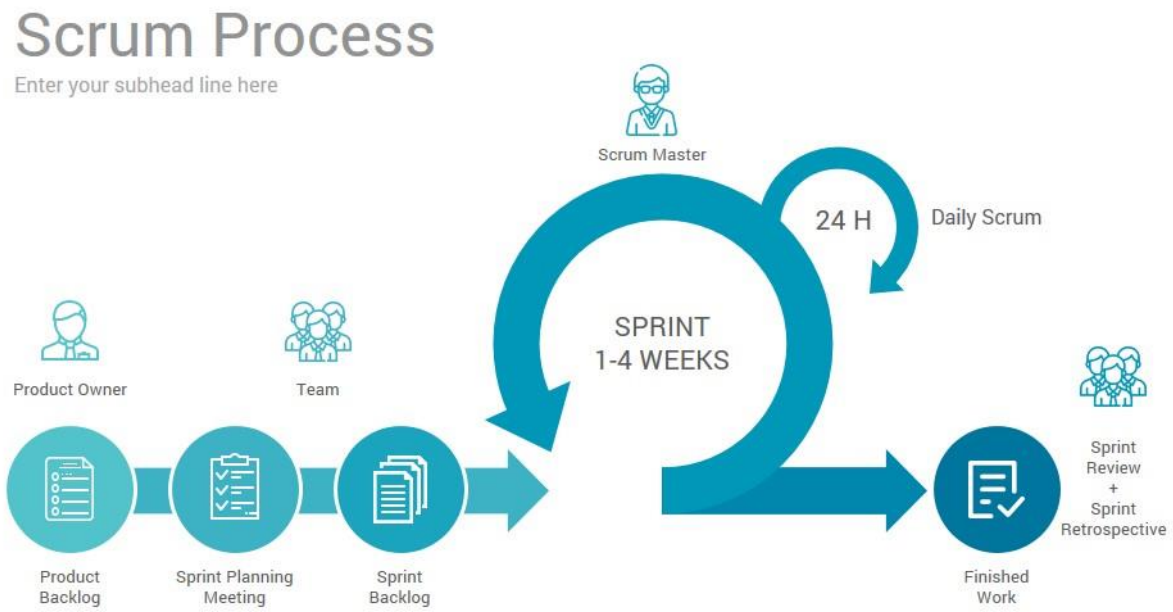
Agile 방법론



Scrum(스크럼)

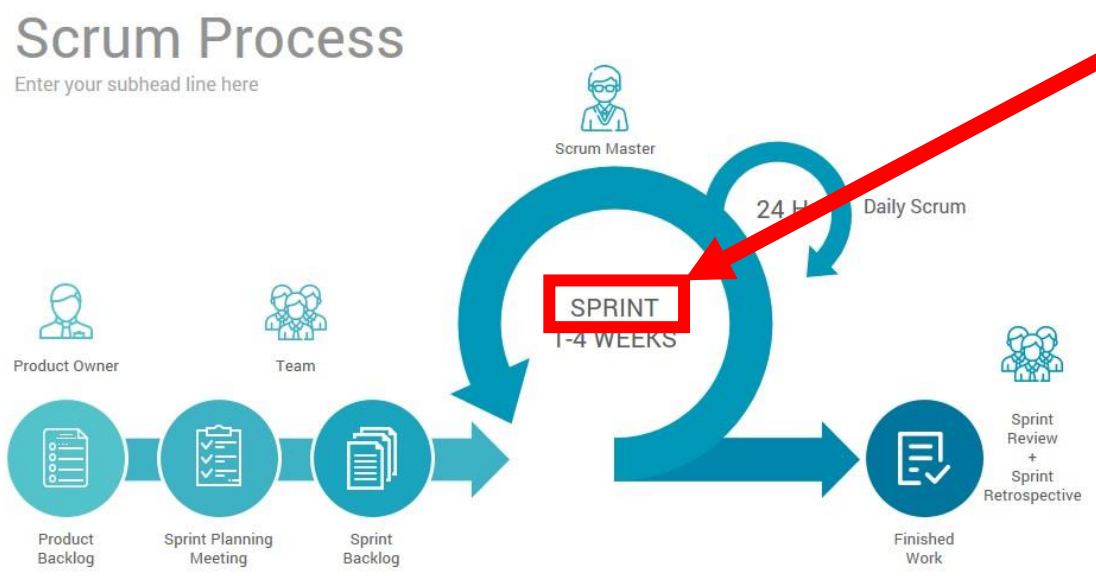
Kanban(칸반)

Scrum(스크럼)



1. 개발자와 고객 사이의 지속적인 커뮤니케이션을 통해 요구사항을 수용
2. 고객이 결정한 사항을 가장 우선적으로 시행
3. 팀원들과 주기적인 미팅을 통해 프로젝트를 점검
4. 주기적으로 제품 시현을 하고 고객으로부터 피드백 수용

Scrum(스크럼)



작은 기능에 대해

"계획, 개발, 테스트, 기능 완료"

에 대해 주기적으로 시행하는 것

일반적인 스프린트 주기 : 1~2주

Kanban(칸반)

장점

- 업무 흐름의 시각화
- 진행 중 업무의 제한
- 명시적 프로세스 정책 수립
- 업무 흐름의 측정과 관리

Git

Git

- Git 이란?
  소스 코드를 효율적으로 관리하기 위해 만들어진 “분산형 버전 관리 시스템”

- 사용 이유?
  소스 코드의 변경 이력을 쉽게 확인
  특정 시점에 저장된 버전과 비교하거나 특정 시점으로 돌아가기 위해

Branch(브랜치)

Branch

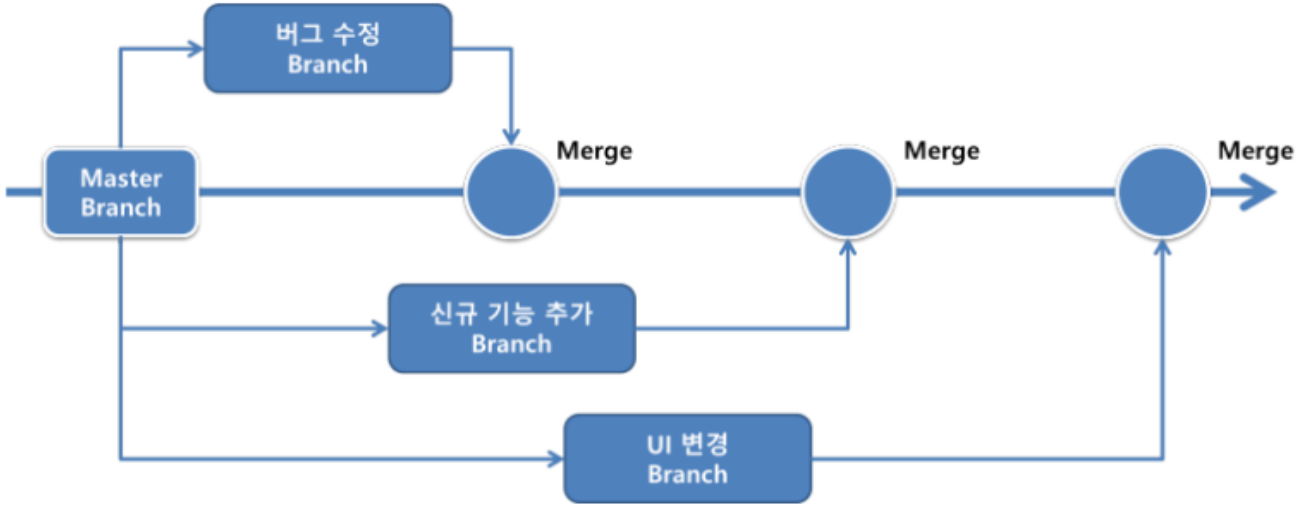
독립적으로 어떤 작업을 하기 위해 필요한 개념

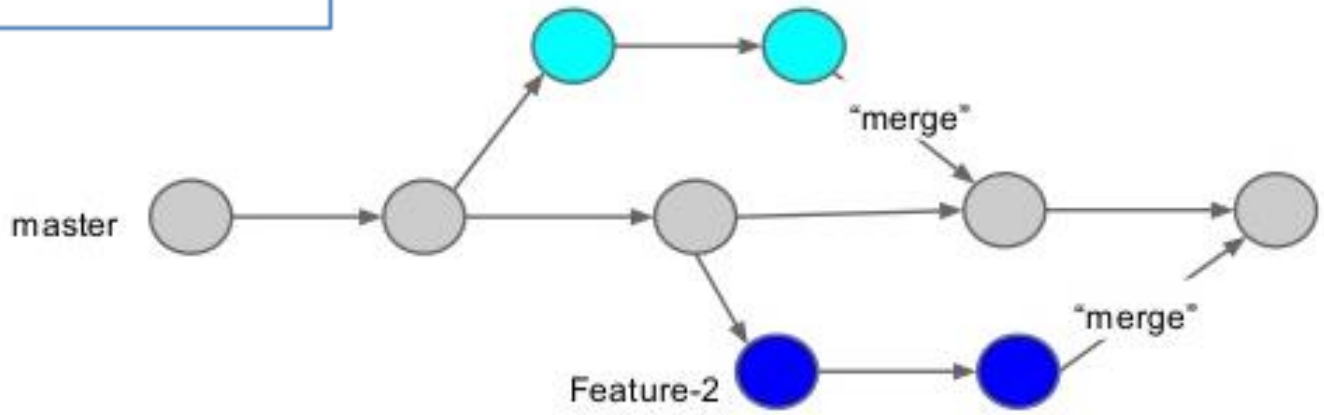
Ex) A라는 사람이 “로그인” 기능을 만들고, B라는 사람이 “버그 수정” 을 할 때

A와 B는 **최초 Branch에서 파생한 각각의 Branch**를 만들어 작업을 진행하고

최초 Branch 로 Merge를 통해 각자가 작업한 것을 합칠 수 있다.

Branch


버그 수정
Branch
Master
Branch
Merge
Merge
Merge
신규 기능 추가
Branch
UI 변경
Branch


"merge"
master
"merge"
Feature-2

Branch 생성하기

```
git branch  # local branch 목록 확인

git branch "브랜치명" # 현재 branch에서 새로운 branch 생성

git checkout "전환 브랜치명" # branch 이동

git branch -d "브랜치명" # bran 삭제
# (단, 삭제할 branch가 현재 branch에 합쳐져 있을 경우에만)
```

새로운 branch 생성 & 이동 동시에

```
git checkout -b "만들 브랜치명"
```

대형 프로젝트를 위해 알아두면 좋은 지식!

Branch의 종류

main(master)

release

develop

hotfix

feature

Branch - main(master)

- 제품으로 출시될 수 있는 브랜치
- 배포(Release) 이력을 관리하기 위해 사용
- 배포 가능한 상태만을 관리하는 브랜치

Branch - develop

- 다음 출시 버전을 개발하는 브랜치
- 기능 개발을 위한 브랜치들을 병합하기 위해 사용
- 평소 개발을 진행하는 브랜치

Branch - feature

- 기능 개발을 진행하는 브랜치
- 새로운 기능 개발 및 버그 수정할 때마다 ‘develop’ 에서 분기
- 공유할 필요가 없어 로컬에서 진행 후 develop 에 merge 해 공유
- 브랜치 이름 : feature/xxx

```
git checkout -b feature/이름 develop

/* 개발~~ */

git checkout develop
git merge feature/이름
git branch -d feature/이름
git push origin develop
```

Branch – release

- 출시 버전을 준비하는 브랜치
- 배포를 위한 전용 브랜치
- 브랜치 이름 : release-0.0

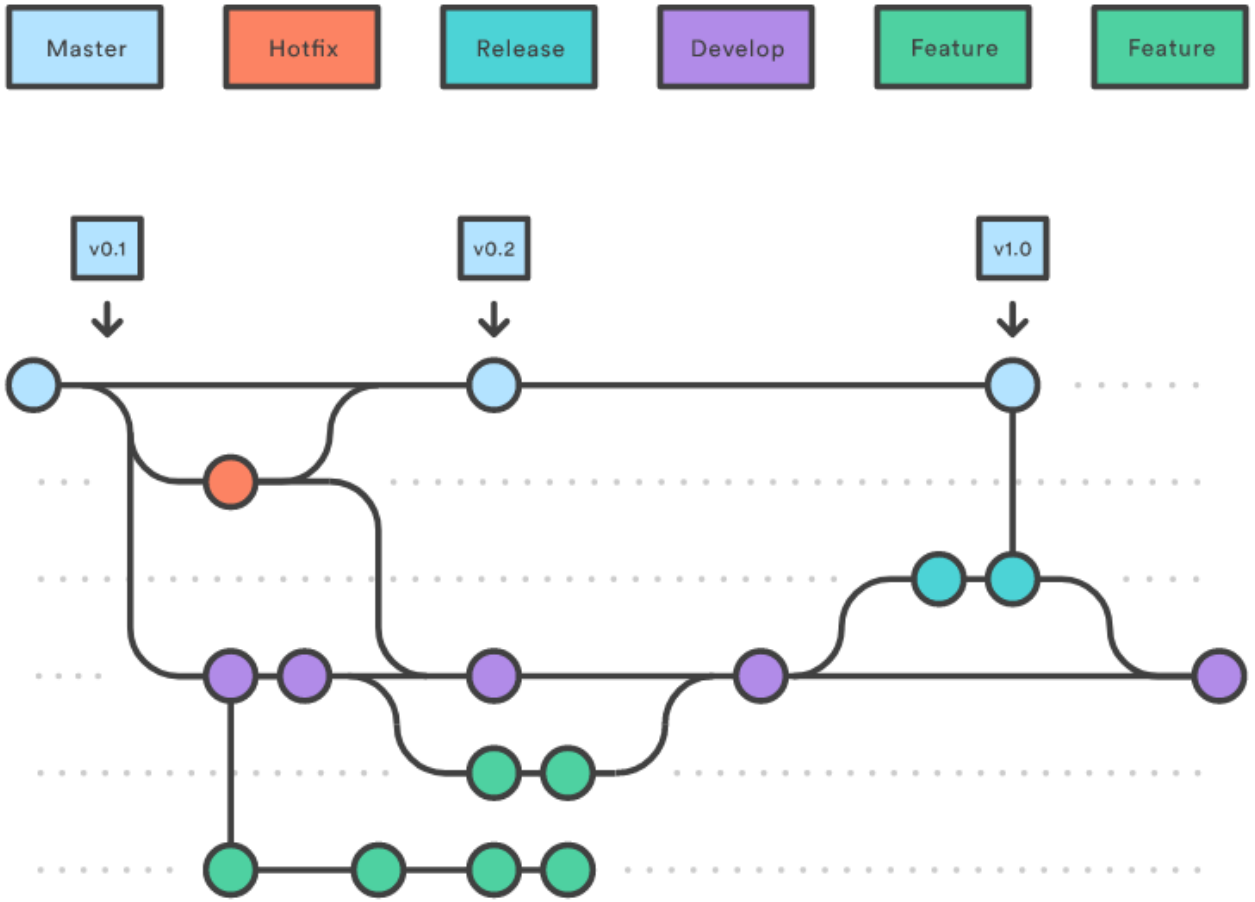
```
git checkout -b release-1.2 develop

/* 배포 사이클 */

git checkout master
git merge release-1.2
git tag -1 1.2

git checkout develop
git merge release-1.2
git branch -d release-1.2
```

Branch – hotfix

- 출시 버전에서 발생한 버그 수정 브랜치
- 배포한 버전에 긴급하게 수정해야 할 필요가 있는 경우 사용
- main(master) 에서 분기
- 브랜치 이름 : hotfix-0.0.0

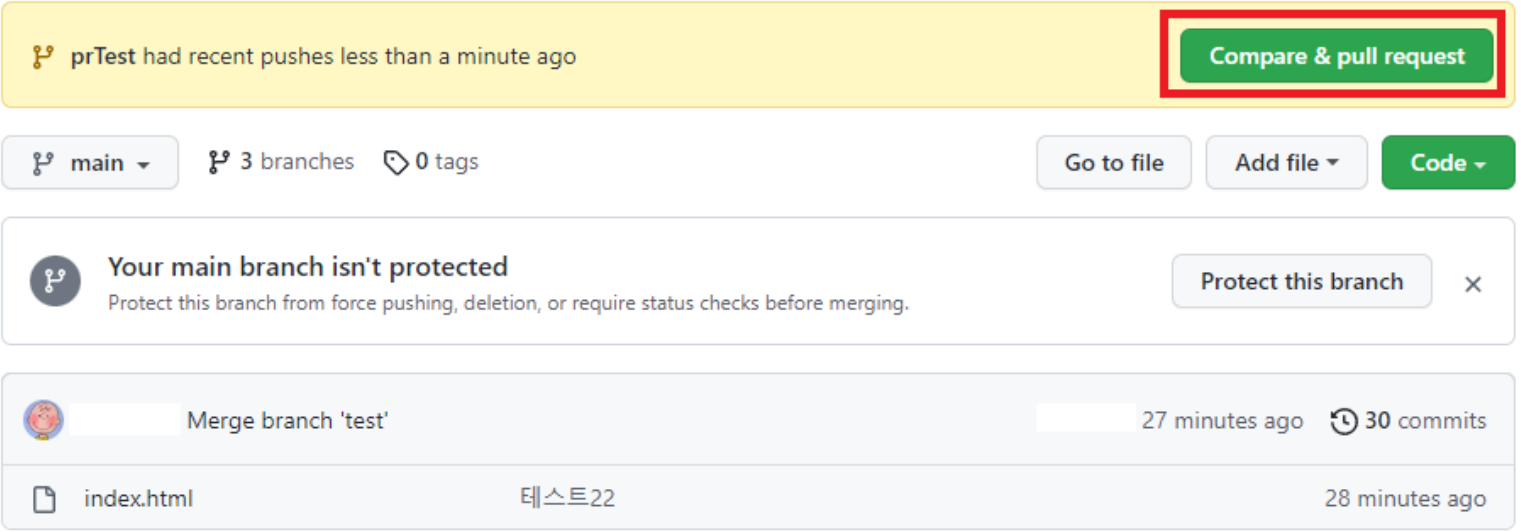
```
git checkout -b hotfix-1.2.1 master

/* 문제 수정 */

git checkout master
git merge hotfix-1.2.1
git tag -a 1.2.1

git checkout develop
git merge hotfix-1.2.1
git branch -d hotfix-1.2.1
```

Branch 종류

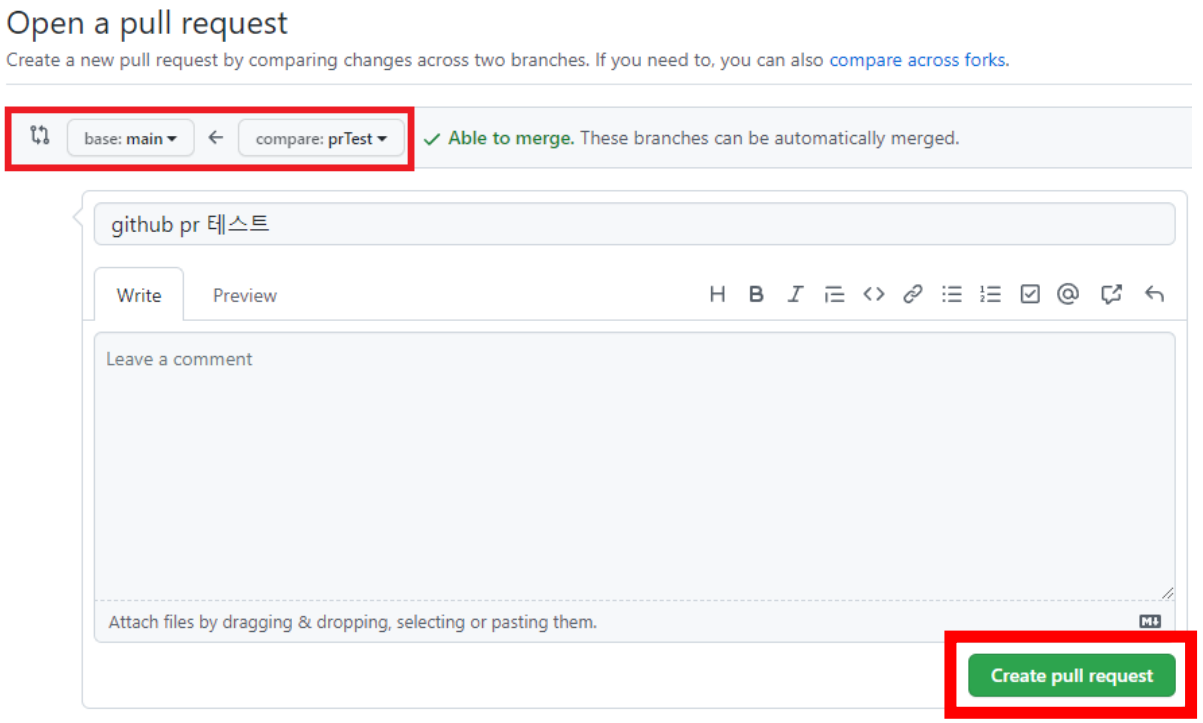


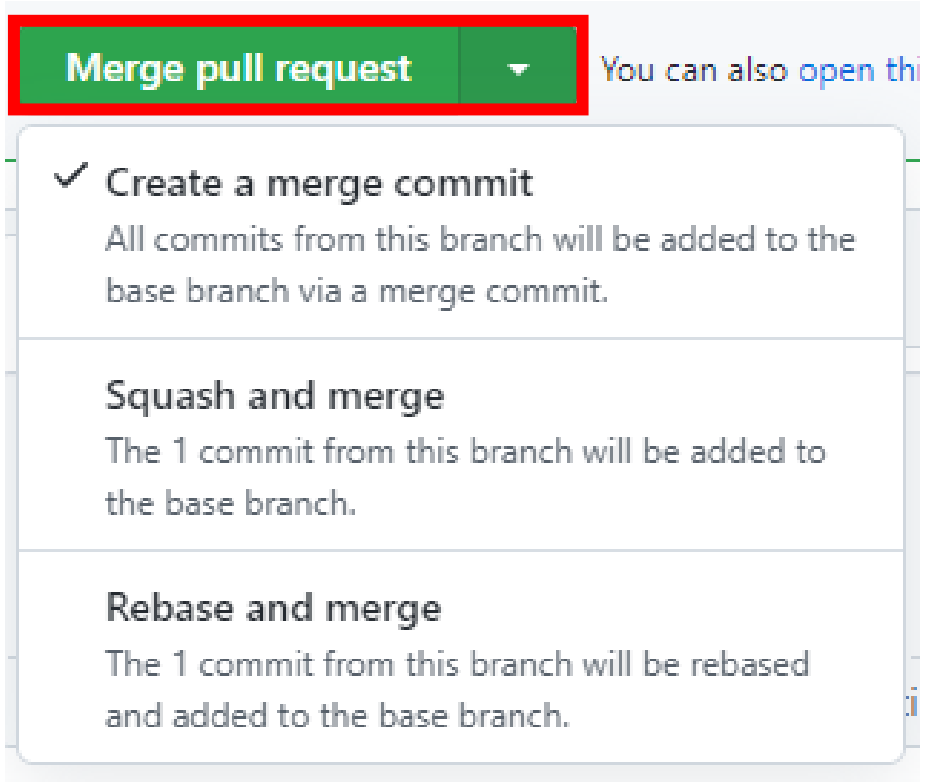
Pull Request

- Push 권한이 없는 오픈 소스 프로젝트에 기여할 때 많이 사용함.
- **“ 내가 수정한 코드가 있으니 내 branch를 가져가 검토 후 병합(merge) 해주세요!! ”**
- 당황스러운 코드 충돌을 줄일 수 있음



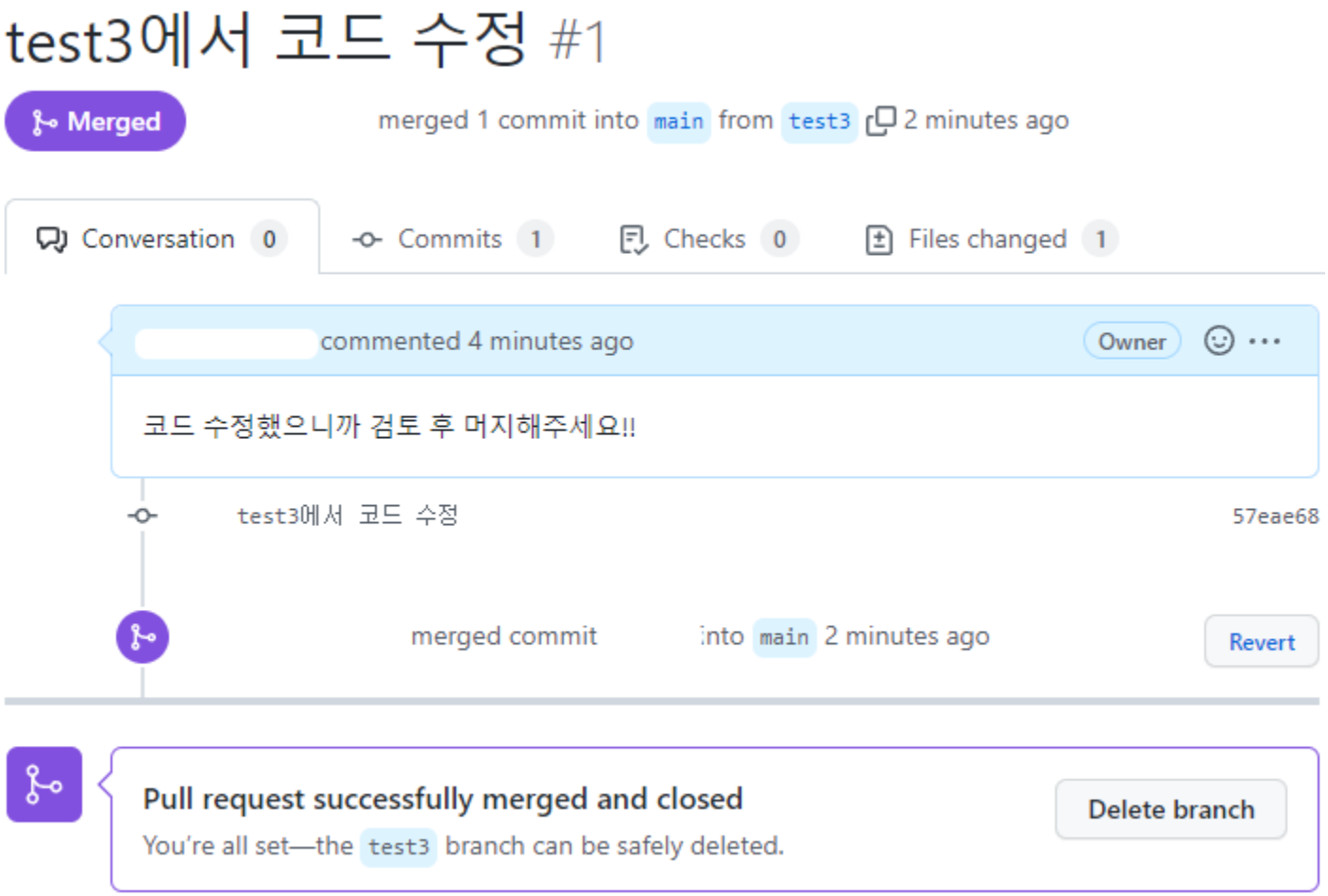
Pull Request





Pull Request

merge 완료!!



성공적인 프로젝트 진행을 위해 반드시 지켜야 할 3가지

1. 일단 Pull 받고 시작하기
2. 일단 Pull 받고 시작하기
3. 일단 Pull 받고 시작하기